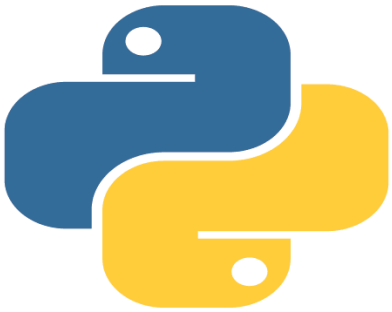

Python

Part 4

Python - Functions

A Python function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing.

Defining a Python Function

Here are simple rules to define a function in Python –

Function blocks begin with the keyword def followed by the function name and parentheses ().

Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.

The first statement of a function can be an optional statement; the documentation string of the function or docstring.

The code block within every function starts with a colon (:) and is indented.

The statement return [expression] exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as return None.

Syntax to Define a Python Function

```
def function_name( parameters ):
   "function_docstring"
   function_suite
   return [expression]
```

Example to Call a Python Function:

```
# Function definition is here
def printme( str ):
   "This prints a passed string into this function"
   print (str)
```

```
    return;

# Now you can call the function
printme("I'm first call to user defined function!")
printme("Again second call to the same function")
```

Output:

```
I'm first call to user defined function!
Again second call to the same function
```

Example:

```
def sum(a,b):
    sum=a+b
    print("sum of two no's:",sum)
sum(2,5)
```

Output:

```
Sum of two no's:7
```

Arbitrary or Variable-length Arguments

You may need to process a function for more arguments than you specified while defining the function. These arguments are called variable-length arguments and are not named in the function definition

Syntax

```
def functionname([formal_args,] *var_args_tuple ):
   "function_docstring"
   function_suite
   return [expression]
```

(*) is placed before the variable name that holds the values of all non-keyword variable arguments

Example:

```
# Function definition is here
def printinfo( arg1, *vartuple ):
   "This prints a variable passed arguments"
   print ("Output is: ")
   print (arg1)
   for var in vartuple:
      print (var)
   return;

# Now you can call printinfo function
printinfo( 10 )
printinfo( 70, 60, 50 )
```

Output:

```
Output is:
10
Output is:
70
60
50
```

Keyword Arguments

Keyword arguments are related to the function calls. When you use keyword arguments in a function call, the caller identifies the arguments by the parameter name

Example:

```
# Function definition is here
def printinfo( name, age ):
   "This prints a passed info into this function"
   print ("Name: ", name)
   print ("Age ", age)
   return;

# Now you can call printinfo function
printinfo( age=50, name="miki" )
```

Output:

```
Name:  miki
Age  50
```

Default Arguments

A **default argument** is an argument that assumes a default value if a value is not provided in the function call for that argument.

Example:

```
def printinfo( name, age = 35 ):

   "This prints a passed info into this function"

   print ("Name: ", name)

   print ("Age ", age)

   return;

# Now you can call printinfo function

printinfo( age=50, name="miki" )
```

```
printinfo( name="miki" )
```

Output:

```
# Function definition is here

Name:  miki

Age  50

Name:  miki

Age  35
```

OOPs(Object oriented programming language)

OOP is an abbreviation that stands for **Object-oriented programming** paradigm. It is defined as a programming model that uses the concept of **objects** which refers to real-world entities with state and behavior. This chapter helps you become an expert in using object-oriented programming support in Python language.

Python is a programming language that supports object-oriented programming. This makes it simple to create and use classes and objects.

Class & Object

A **class** is an user-defined prototype for an object that defines a set of attributes that characterize any object of the class. The attributes are data members (class variables and instance variables) and methods, accessed via dot notation.

An **object** refers to an instance of a certain class. For example, an object named **obj** that belongs to a class **Circle** is an instance of that class. An object comprises both data members (class variables and instance variables) and methods.

Concept of class and objects in real world

Example of class is bird,human,animal like that from the example bird is a class.In this class have property and behavior

The property of this class is name, color, legs ,wings

The behavior of class is fly, sing pattern, thinking types, drinking of water

From the example class is human

It have behavior and property

The property of this class is name, height,weight,personal details

The behavior of the class is walking,running,and doing programs

Python - OOP Concepts

In the real world, we deal with and process objects, such as student, employees, invoice, car, etc. Objects are not only data and not only functions, but combination of both. Each real-world object has attributes and behavior associated with it.

Attributes

Name, class, subjects, marks, etc., of student

Name, designation, department, salary, etc., of employee

Behavior

Processing attributes associated with an object.

Principles of OOPs Concepts

Object-oriented programming paradigm is characterized by the following principles −

- Class
- Object
- Encapsulation
- Inheritance
- Polymorphism

Example:

```
class Car:
```

```
    def __init__(self, name, color, version):
        self.name = name
        self.color = color
        self.version = version
    def show_details(self):
        print('Name:',self.name, ‘color:', self.color, ‘Version:',    self.version)
c=Car(“Bmw”,”Black”,”n6”)
c.show_details()
```

Output:

```
Name:BMW color:Black,Version:n6
```

Example:

Add features another cars

```
class Car:
    def __init__(self, name, color, version):
        self.name = name
        self.color = color
        self.version = version
    def show_details(self):
        print('Name:',self.name, ‘color:', self.color, ‘Version:',    self.version)
c1=Car(“Bmw”,”Black”,”n6”)
c2=Car(“Honda”,”red”,”v6”)
c1.show_details()
c2.show_details()
```

Output:

```
Name:BMW color:Black,Version:n6
Name:Honda color:Red,Version:v6
```

__init__()

Is a special method that initializes an individual object.This method run automatically each time an object of a class is created.__init__() method is generally used to perform operations that are necessary before the object are create

Self

__init__() any number of parameters butfirst parameter will always variable called self. When create a new class instance python automatically pass the instance to self parameter in __init__()

Modification of class

For change the data in class

Example:

```
class Car:
    def __init__(self, name, color, version):
        self.name = name
        self.color = color
        self.version = version
    def show_details(self):
        print('Name:',self.name','color:',self.color,
'Version:',    self.version)
 c1=Car("Bmw","Black","n6")
 c2=Car("Honda","red","v6")
 c2.version=v7
 c1.show_details()
 c2.show_details()
```

Output:

```
Name:BMW color:Black,Version:n6
Name:Honda color:Red,Version:v7
```

Delete operation

Example:

```
class Personaldetail:
    def __init__(self, name, age,qualification):
        self.name = name
        self.age = age
        self.qualification = qualification
    def show(self):
        print("name is", self.name, "age  is",self.age
,"qualification",self.qualification)
obj = Personaldetail("Maya", 20,"Btech")
del obj
obj.show()
```

Output:

```
NameError: name 'obj' is not defined
```

Pass statement in class

Example:

```
class Demo:
        pass
```

Output:

```
>>>
```

Python inheritance

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a class (called a child or derived class) to inherit attributes and methods from another class (called a parent or base class). In this article, we'll explore inheritance in Python.

Parent class is the class being inherited from, also called base class.

Child class is the class that inherits from another class, also called derived class.

Syntax for Inheritance

```
class ParentClass:
    # Parent class code here
    pass

class ChildClass(ParentClass):
    # Child class code here
    pass
```

Explanation of Syntax

Parent Class:

- This is the base class from which other classes inherit.
- It contains attributes and methods that the child class can reuse.

Child Class:

- This is the derived class that inherits from the parent class.
- Syntax for inheritance is class ChildClass(ParentClass).
- Child class automatically gets all attributes and methods of the parent class unless overridden.

Creating a Parent Class

In object-oriented programming, a parent class (also known as a base class) defines common attributes and methods that can be inherited by other classes. These attributes and methods serve as the foundation for the child classes. By using inheritance, child classes can access and extend the functionality provided by the parent class.

Creating a Child Class

A child class (also known as a subclass) is a class that inherits properties and methods from its parent class. The child class can also introduce additional attributes and methods, or even override the ones inherited from the parent.

subclass has been made is called a **superclass**

Other names of **superclass** are **base class** or **parent class**

other names of **subclass** are **derived class** or **child class**

The process of creating a subclass of a class is called inheritance.

All the attributes and methods of superclass are inherited by its subclass

Example:

```
class Animal: # super class
        def eat(self):
        print("I can eat")
class cat(Animal): # subclass
        def walk(self):
        print("I walking")
Animal_io=cat()
Animal_io.eat()
```

Output:

```
I can eat
I walking
```

Example:

```
class Person():
        def display1(self):
        print("This is superclass")
class Employee(Person):
        def display2(self):
        print("This is subclass")
emp = Employee()
```

```
emp.display1()
emp.display2()
```

Output:

```
This is superclass
This is subclass
```

Example:

```
# parent class
class Person:

        # __init__ is known as the constructor
        def __init__(self, name, department):
            self.name = name
            self.department = department

        def display(self):
            print(self.name)
            print(self.department)

# child class
class Employee(Person):
        def __init__(self, name,department, salary, post):
            self.salary = salary
            self.post = post

            # invoking the __init__ of the parent class
            Person.__init__(self, name, department)

# creation of an object variable or an instance
a = Employee('Adharsh', "pwd", 200000, "LDC")

# calling a function of the class Person using its instance
a.display()
```

Output:

```
Adarsh
Pwd
```

The super() function is used to give access to methods and properties of a parent or sibling class.
The super() function returns an object that represents the parent class.

Syntax:

```
super()
```

Example:

```
class Emp():
    def __init__(self, id, name, Add):
        self.id = id
        self.name = name
        self.Add = Add

# Class freelancer inherits EMP
class Freelance(Emp):
    def __init__(self, id, name, Add, Emails):
        super().__init__(id, name, Add)
        self.Emails = Emails

Emp_1 = Freelance(103, "Suraj kr gupta", "Noida","KKK@gmails")
print('The ID is:', Emp_1.id)
print('The Name is:', Emp_1.name)
print('The Address is:', Emp_1.Add)
print('The Emails is:', Emp_1.Emails
```

Output:

```
Suraj kr gupta
Noida
KKK@gmails
```

Polymorphism of class

The word polymorphism means having many forms. In programming, polymorphism means the same function name (but different signatures) being used for different types. The key difference is the data types and number of arguments used in function.

Built in polymorphism

```
# len() being used for a string
print(len("geeks"))
# len() being used for a list
print(len([10, 20, 30]))
```

Polymorphism with class methods:

The below code shows how Python can use two different class types,

Example:

```
class tvm:
        def weather(self):
            print("weather of tvm")

        def place(self):
            print("capital city of kerala is tvm")
class kochi:
        def weather(self):
            print("weather of kochi ")
```

```
        def place(self):
            print("queen of arabian sea")
a = tvm()
b = kochi()
for x in a,b:
        x.weather()
        x.place()
```

Output:

```
weather of tvm
capital city of kerala is tvm
weather of kochi
queen of arabian sea
```

Operator polymorphism

Operator polymorphism in Python, also known as operator overloading, refers to the ability of a single operator symbol to perform different operations depending on the data types of the operands it acts upon.

Example:

```
a=1
b=4
Print(a+b)
a="Hello"
b="world"
print(a+b)
```

Output:

```
5
Helloworld
```

Abstraction of class

- Data abstraction means showing only the essential features and hiding the complex internal details.
- Technically, in Python abstraction is used to hide the implementation details from the user and expose only necessary parts, making the code simpler and easier to interact with.

A smartphone is a great real-life example of data abstraction you can make calls or take photos without knowing how signals or storage work. Only essential features are shown, complex details are hidden.

Abstract classes are created using **abc module** and **@abstractmethod** decorator, allowing developers to enforce method implementation in subclasses while hiding complex internal logic.

Example:

```
from abc import ABC, abstractmethod

class Greet(ABC):

    @abstractmethod

    def say_hello(self):

            print(“hello world”)

class English(Greet):

    def say_hello(self):

        return "Hello!"

g = English()

print(g.say_hello())
```

Output:

```
Hello!
```